MapReduce: Simplified Data Processing on large clusters. Response Xiyao Long

The MapReduce system abstract the primitive of “map/reduce” operation in the typical functional programming language and use the “map” operation to compute sets of intermediate “key/value” pairs for data records, and apply a reduce operation to the intermediate key/value pairs of each data set. The program administers M partition on the input data and R partitioning on the key space. Based on this partition scheme, the master needs to assign work of each partition to different workers. The intermediate key-value pairs resulting from the map function can be stored either in memory or on disk but the disk address will be sent to the master so then they can be forwarded to corresponding “reduce” operation workers.
The fault tolerance in the system is partially enabled by the fact that the master would label the status of possible worker machines including labeling a machine as “failed” if no response is received after a certain time.

The master will prompt the re-execution of the “Map” operation of the partition corresponding to a failed machine because the task data are no longer accessible. The master will also notify all workers to “access the result of the new execution. The system uses atomic filename modifying operation to keep track of which “map-reduce” operation has finished.

The paper also talked about the bounds of M and R and pointed out that memory usage for the master to keep the O(M*R) states is pretty small. But in practice user choose M and R based on file size and the number of worker machines.

The MapReduce implementation allows much easier manifestation of the parallel data processing scheme as the algorithm already considers the handling of machine failure and slow machines which are very common in a large distributed system.

I think this scheme is restricted in the sense that it handles problems that are parallelizable if the data are dependent and we can not extract parallel process which enables us to segment the data into M sections, the algorithm lost its efficiency. However, considering that the algorithm is design for large distributed systems with huge amount of data, it’s pretty unlikely that a problem driven by the available large data size does not include some parallelizable data processing steps.